ISO/IEC JTC1 SC22 WG21 P2795R4
Date: 2023-11-10
To: SG12, SG23, EWG, CWG
Thomas Köppe <tkoeppe@google.com>

Erroneous behaviour for uninitialized reads

Contents

Revision history

- P2795R0: Initial revision. This revision was presented to SG23 and to EWG in Issaquah.
- P2795R1: Changed title and revised presentation to focus on uninitialized variables, and extract erroneous behaviour in general only as a future direction. This revision was presented to EWG in Varna.
- P2795R2: Reviewed by CWG and considered done as far as the present proposal goes, but CWG has requested that an opt-out mechanism be included in the proposal.
- P2795R3: The target of the modified behaviour has been changed from “object with automatic storage duration” to “non-static local variable”. This excludes temporary objects and function parameters, which proved difficult to handle. Also, an opt-out mechanism is added via an attribute on variable definitions (backward-compatible with C++11).
- P2795R4: This version. The target of the modified behaviour has been restored to once again be *all* objects with automatic storage duration and temporaries. (However, there is no opt-out mechanism for temporary objects.) Wording has been added for `std::bit_cast` to handle erroneous values. It has been clarified that erroneously initialized storage can still result in undefined behaviour if the object representation is not valid for the type (e.g. for `bool`). It has been clarified that the opt-out attribute on a function parameter has to appear on the function's first declaration. The wording has been updated to establish a standard phrasing used to define situations that have erroneous behaviour.

Summary

We propose to address the safety problems of reading a default-initialized automatic variable (an "uninitialized read") by adding a novel kind of behaviour for C++. This new behaviour, called *erroneous behaviour*, allows us to formally speak about "buggy" (or "incorrect") code, that is, code that does not mean what it should mean (in a sense we will discuss). This behaviour is both "wrong" in the sense of indicating a programming bug, and also well-defined in the sense of not posing a safety risk.

Motivation

Pragmatically, there are very few C++ programs in the real world that are entirely correct. In terms of the Standard, that means most programs are not constrained by the specification at all, since they run into undefined behaviour. This is ultimately not very helpful to real software development efforts. The term "safety" has been mentioned as a concern in both C and C++, but it is a nebulous and slippery term that means different things to different people. A useful definition that has come up is that "safety is about the behaviour of incorrect programs".

The motivating example of unsafe code that we address in this proposal is reading a default-initialized variable of automatic storage duration and scalar type:

Example M:

```
extern void f(int);

int main() {
  int x;     // default-initialized, value of x is indeterminate
  f(x);      // glvalue-to-prvalue conversion has undefined behaviour
}
```

This code is blatantly *incorrect*, but it occurs commonly as a programming error. The code is also *unsafe* because this error is exploitable and leads to real, serious vulnerabilities.

With increased community interest in safety, and a growing track record of exploited vulnerabilities stemming from errors such as this one, there have been calls to fix C++. The recent P2723R1 proposes to make this fix by changing the undefined behaviour into well-defined behaviour, and specifically to well-define the initialization to be zero. We will argue below that such an expansion of well-defined behaviour would be a great detriment to the understandability of C++ code. In fact, if we want to both preserve the expressiveness of C++ and also fix the safety problems, we need a novel kind of behaviour.

The excellent survey paper P2754R0 analyses a number of possible changes to automatic variable initialization. We had circulated the core idea of proposed erroneous behaviour on the reflector previously, and that option is contained in the survey. We reproduce the survey summary here, with minor modifications and added colour:

Proposed Solution	Viability	Backward Compatibility	Expressibility
Always Zero-Initialize	Viable	Compatible	**Worse**
Zero-Initialize or Diagnose	**Unclear**	Correct-Code Compatible	Unchanged
Force-Initialize in Source	Viable	**Incompatible**	Better
Force-Initialize or Annotate	Viable	**Incompatible**	Better
Default Value, Still UB	**Nonviable**	Compatible	Unchanged
Default Value, Erroneous	Viable	Compatible	Unchanged

Proposed Solution	Viability	Backward Compatibility	Expressibility
Value-Initialize Only	**Unclear**	**Unclear**	**Unclear**

Conclusion from P2754R0

The introduction of a novel notion of erroneous behaviour is the only solution that is viable, compatible with existing code, and that does not sacrifice expressiveness of the language. (A detailed discussion of expressiveness and the meaning of code follows below.) This leads us to our main proposal.

Proposal: reading an uninitialized variable is erroneous

We propose to change the semantics of reading an uninitialized variable:

> Default-initialization of an automatic-storage object initializes the object with a **fixed value defined by the implementation**; however, **reading that value is a conceptual error**. Implementations are **allowed and encouraged to diagnose this error**, but they are also allowed to ignore the error and **treat the read as valid**. Additionally, an **opt-out mechanism** (in the form of an attribute on a variable definition or function parameter) is provided to restore the previous behaviour.

This is a novel kind of behaviour. Reading an uninitialized value is never intended and a definitive sign that the code is not written correctly and needs to be fixed. At the same time, we do give this code well-defined behaviour, and if the situation has not been diagnosed, we want the program to be stable and predictable. This is what we call *erroneous behaviour*.

In other words, it is still an "wrong" to read an uninitialized value, but if you do read it and the implementation does not otherwise stop you, you get some specific value. In general, implementations must exhibit the defined behaviour, at least up until a diagnostic is issued (if ever). There is no risk of running into the consequences associated with undefined behaviour (e.g. executing instructions not reflected in the source code, time-travel optimisations) when executing erroneous behaviour.

Here is a comparison of the status quo, the proposal P2723R1, and this proposal, with regards to the above Example M.

C++23	P2723R1 (default-init zero)	This proposal
undefined behaviour	well-defined behaviour	erroneous behaviour
definitely a bug	may be intentionally using 0, or may be a bug	definitely a bug
common compilers allow rejecting (e.g. `-Werror`), this selects a non-conforming compiler mode	conforming compilers cannot diagnose anything	conforming compilers generally have to accept, but can reject as QoI in non-conforming modes

Comparison of Example M under various proposals

A change from the earlier revision P2795R0 is that the permission for an implementation to reject a translation unit “if it can determine that erroneous behaviour is reachable within that translation unit” has been removed: Richard Smith pointed out that such a determination is not generally possible. Any attempt to reject any erroneous behaviour at all would most likely have false positives, since it is in general

impossible to determine whether a particular piece of code ends up being used. Whereas undefined behaviour in unused code would not currently prevent a build from succeeding, it could be rather disruptive if that code would now be rejected for containing erroneous behaviour, even if it was never used. Therefore, in this revision we leave it as pure QoI whether implementations attempt to detect that erroneous behaviour might be encountered and issue appropriate warnings or errors, similar to how implementations currently attempt to warn about undefined behaviour (e.g. with the `-Wuninitialized` flag).

In R3 we had changed the proposal to only target *variables* with automatic storage duration, whereas R2 and this R4 again target *all* objects with automatic storage duration. The latter include temporary objects (probably, though this is subject to CWG 365 and CWG 1634) and function parameters. For R3, we have found it difficult to provide adequate opt-out mechanisms for non-variable objects, whereas an opt-out mechanism for local variables is straight-forward (see below). However, EWG has clarified that *all* automatic-storage objects should be treated the same, both in order to keep the mental model simple and to provide the safety benefits in all cases. Applying the opt-out mechanism to function parameters should be possible after all, but none will be provided for temporary objects. However, temporary objects are much less likely to be observable, so that an optimizing compiler should rarely need to create additional memory writes to implement the proposed behaviour.

Note that we do not want to mandate that the specific value actually be zero (like P2723R1 does), since we consider it valuable to allow implementations to use different “poison” values in different build modes. Different choices are conceivable here. A fixed value is more predictable, but also prevents useful debugging hints, and poses a greater risk of being deliberately relied upon by programmers.

Performance and security implications

During core wording review, we noted a number of implications of changing initialization semantics, which we want to call out explicitly here.

- The automatic storage for an automatic variable is always fully initialized, which has potential performance implications. P2723R1 discusses the costs in some detail. Note that this cost even applies when a class-type variable is constructed that has no padding and whose default constructor initializes all members.
- In particular, unions are fully initialized. Copying a union is not erroneous, and in general, copying padding bits is not erroneous. In detail, this implies that glvalue-to-prvalue conversion of erroneous values is not itself erroneous, but doing anything with such a value other than copying it is erroneous. This is entirely parallel for the current undefined behaviour rules around indeterminate values.
- This proposal affects only the semantics of the initializaton of variables, not of all uses of indeterminate values in general. For example, one can copy an indeterminate value into an initialized variable, and reading that value can still lead to undefined behaviour, notwithstanding the variable’s well-intentioned initialization.
- The proposed changes to the initialization rules only affect the initialization of an automatic variable as a single operation. Here is example that is *not* affected by this proposal and that involves an automatic variable and default-initialization separately:

  ```
  void f() {
    char data[] = {'s', 'e', 'c', 'r', 'e', 't'};   // automatic variable
    ::new (static_cast<void*>(data)) char;          // default-initialization
  }
  ```

  The placement-new does *not* guarantee to overwrite the data in its storage.
- Function parameters and temporary objects are also affected by this proposal.

An opt-out mechanism

Motivation

The proposed change of behaviour has a runtime cost for existing code, since in general additional initialization of memory is now required. We reiterate at this point that the proposed change is nothing more and nothing less than a safety and security feature: It does not affect the semantics of correct code, and it does not alter the meaning of correct code or affect whether code is correct.

Users who do not wish to accept this performance penalty should be given an option to disable this new safety feature and thereby recover the previous C++23 behaviour. Concretely, reading from a variable that has been opted out of the "erroneous behaviour" initialization has undefined behaviour again (so the compiler is allowed to assume that this does not happen and optimise accordingly).

A word of caution

Before we go into the details of the opt-out syntax, we have to discuss a potential pitfall (pointed out by the indefatigable Richard Smith): Users may wish to annotate their code to be explicit about the fact that they do not intend to initialize a variable. This is entirely unrelated to the safety feature of erroneous initialization. However, there is a danger that the opt-out for the safety feature is mistaken for a mechanism to document intention. If, for example, we made a syntax `= noinit` available, users might be tempted to use this to document their code:

hypothetical counter-example, inappropriate use

```
int x;            // C++23, legacy
int x = noinit;   // same thing, modern replacement, self-documenting in C++26
```

It is plausible that a modern codebase would adopt a rule that initializers must never be omitted (as in `int x;`). Users might then expect that the new syntax would provide a principled alternative. (This was proposed, for example, in P0632R0.) But instead they would unwittingly and unintentionally opt out of a safety mechanism.

Decision

We propose to use an *attribute* to request opting out of the new behaviour. Attributes are a familiar mechanism in C++, and attributes are allowed in the two locations where we need it, namely appertaining to a variable definition and to a function parameter. An attribute works well semantically, since the sole purpose of this attribute is to disable a safety mechanism. Code that is correct with the attribute is also correct if the attribute is removed or ignored. Indeed, the attribute makes code strictly less permissive.

The attribute is allowed only on variable definitions and on function parameters. It can appear either at the beginning of a declaration (in which case it applies to all declarators), or on an individual declarator immediately after the *declarator-id*. It is ill-formed to place the attribute anywhere else.

No opt-out mechanism is provided for temporary objects, since we have not been able to find a suitable syntactic location. However, we consider this only a minor problem, since it is comparatively much harder to observe temporary objects, and if an optimizing compiler can prove that the storage is not observed, it can plausibly avoid dead stores.

Alternative, rejected approaches and previous discussions and polls are discussed below. Note that unlike the alternative suggestions, attributes are backward-compatible with C++11.

It remains to decide the name of the attribute. In light of the word of caution above, we would like to stay clear of the much-suggested term “uninitialized”. The opt-out is expected to be an expert-only feature that disables a safety guardrail and would be used only when performance concerns warrant it. We consider it acceptable for the name to be long and unwieldy, and it is perhaps even a desirable feature for the attribute to *not* appeal to the regular user for a mistaken purpose, as discussed above. We propose the spelling `[[indeterminate]]`. This seems to describe the effect reasonably well and is not prone to being misused to document intentional lack of initialization. To witness this in action:

```
int x [[indeterminate]];
std::cin >> x;

[[indeterminate]] int a, b[10], c[10][10];
compute_values(&a, b, c, 10);

int d[3] = {}, e [[indeterminate]] [3];   // [sic!]
copy_three_values(/*from=*/d, /*to=*/e);

// This class benefits from avoiding determinate-storage initialization guards.
struct SelfStorage {
  std::byte data_[512];
  void f();   // uses data_ as scratch space
};

SelfStorage s [[indeterminate]];   // documentation suggested this

void g([[indeterminate]] SelfStorage s = SelfStorage());   // same; unusual, but
plausible
```

Proposed wording

To establish the meaning of erroneous behaviour, first add an entry to [3, intro.defs]:

> **3.? [defns.erroneous]**
>
> **erroneous behavior**
>
> well-defined behavior that the implementation is recommended to diagnose
>
> [*Note 1 to entry*: Erroneous behavior is always the consequence of incorrect program code. Implementations are allowed, but not required, to diagnose it ([4.1.1, intro.compliance.general]). Evaluation of a constant expression ([7.7, expr.const]) never exhibits behavior specified as erroneous in [4, intro] through [15, cpp]. — *end note*]

Also modify the entry in [3, intro.defs] for “undefined behavior” to free up the term “erroneous”:

> **3.65 [defns.undefined]**
>
> **undefined behavior**
>
> behavior for which this document imposes no requirements
>
> [*Note 1 to entry*: Undefined behavior may be expected when this document omits any explicit definition of behavior or when a program uses an ~~erroneous~~incorrect construct or ~~erroneous~~invalid

> data. Permissible undefined behavior ranges from ignoring the situation completely with unpredictable results, to behaving during translation or program execution in a documented manner characteristic of the environment (with or without the issuance of a diagnostic message), to terminating a translation or execution (with the issuance of a diagnostic message). Many ~~erroneous~~incorrect program constructs do not engender undefined behavior; they are required to be diagnosed. […] — *end note*]

Update the footnote in [4.1.1, intro.compliance.general]p(2.1):

> Although this document states […]
>
> - If a program contains no violations of the rules in [Clause 5, lex] through [Clause 33] and [Annex D, depr], a conforming implementation shall, within its resource limits as described in [Annex B, implimits], accept and correctly execute[Footnote: “Correct execution” can include undefined behavior and erroneous behavior, depending on the data being processed; see [intro.defs] and [intro.execution].] that program.
> - If a program contains a violation […]

Then, define erroneous behaviour; modify [4.1.2, intro.abstract] paragraph 5 and append two new paragraphs:

> 5. A conforming implementation executing a well-formed program shall produce the same observable behavior as one of the possible executions of the corresponding instance of the abstract machine with the same program and the same input. However, if any such execution contains an undefined operation, this document places no requirement on the implementation executing that program with that input (not even with regard to operations preceding the first undefined operation). If the execution contains an operation specified as having erroneous behavior, the implementation is permitted to issue a diagnostic and is permitted to terminate the execution at an unspecified time after that operation.
>
> ?. *Recommended practice*: An implementation should issue a diagnostic when such an operation is executed.
>
> ?. [*Note ?*: An implementation can issue a diagnostic if it can determine that erroneous behavior is reachable under an implementation-specific set of assumptions about the program behavior, which can result in false positives. — *end note*]

Exclude erroneous behaviour from constant expressions; modify [7.7, expr.const] paragraph 5 as follows:

> - […]
> - an expression that would exceed the implementation-defined limits (see [implimits]);
> - an operation that would have undefined or erroneous behavior as specified in [intro] through [cpp], excluding [dcl.attr.assume];[footnote]
> - an lvalue-to-rvalue conversion([conv.lval]) unless […]
> - […]

Finally, make uninitialized reads erroneous. This requires changing both initialization and glvalue-to-prvalue conversion. Modify [6.7.4, basic.indet] heading and paragraph 1:

> **6.7.4 Indeterminate and erroneous values [basic.indet]**
>
> 1. When storage for an object with automatic or dynamic storage duration is obtained, the ~~object has an *indeterminate value*~~bytes comprising the storage for the object have the following initial value:
>
> - If the object has dynamic storage duration, or is the object associated with a variable or function parameter whose first declaration is marked with the `[[indeterminate]]` attribute [9.12.?,

dcl.attr.indeterminate], the bytes have *indeterminate values*,
- otherwise, the bytes have *erroneous values*.

~~, and if~~If no initialization is performed for ~~the~~an object (including for subobjects), ~~that object~~such a byte retains ~~an indeterminate~~its initial value until that value is replaced ([dcl.init.general, expr.ass]). If any bit in the value representation has an indeterminate value, the object has an *indeterminate value*; otherwise, if any bit in the value representation has an erroneous value, the object has an *erroneous value* ([conv.lval]).

[*Note ?*: Objects with static or thread storage duration are zero-initialized, see [basic.start.static]. — *end note*]

2. ~~If~~Except in the following cases, if an indeterminate value is produced by an evaluation, the behavior is undefined~~ except in the following cases~~, and if an erroneous value is produced by an evaluation, the behavior is erroneous and the result of the evaluation is a value determined by the implementation independent of the state of the program:

- If an indeterminate or erroneous value of unsigned ordinary character type ([basic.fundamental]) or `std::byte` type ([cstddef.syn]) is produced by the evaluation of:
  - the second or third operand of a conditional expression ([expr.cond]),
  - the right operand of a comma expression ([expr.comma]),
  - the operand of a cast or conversion ([conv.integral, expr.type.conv,expr.static.cast,expr.cast]) to an unsigned ordinary character type or `std::byte` type ([cstddef.syn]), or
  - a discarded-value expression ([expr.context]),

  then the result of the operation is an indeterminate value or that erroneous value, respectively.
- If an indeterminate or erroneous value of unsigned ordinary character type or `std::byte` type is produced by the evaluation of the right operand of a simple assignment operator ([expr.ass]) whose first operand is an lvalue of unsigned ordinary character type or `std::byte` type, an indeterminate value or that erroneous value, respectively, replaces the value of the object referred to by the left operand.
- If an indeterminate or erroneous value of unsigned ordinary character type is produced by the evaluation of the initialization expression when initializing an object of unsigned ordinary character type, that object is initialized to an indeterminate value or that erroneous value, respectively.
- If an indeterminate or erroneous value of unsigned ordinary character type or `std::byte` type is produced by the evaluation of the initialization expression when initializing an object of `std::byte` type, that object is initialized to an indeterminate value or that erroneous value, respectively.

[*Example ?*:

```
int f(bool b) {
  unsigned char *c = new unsigned char;
  unsigned char d = *c;  // OK, d has an indeterminate value
  int e = d;             // undefined behavior
  return b ? d : 0;      // undefined behavior if b is true
}

int g(bool b) {
  unsigned char c;
  unsigned char d = c;   // OK, d has an erroneous value
  int e = d;             // OK, erroneous behavior
```

```
  return b ? d : 0;       // OK, erroneous behavior if b is true
}
```

— *end example*]

Modify [7.3.2, conv.lval]:

> The result of the conversion is determined according to the following rules:
>
> - […]
> - Otherwise, the object indicated by the glvalue is read ([defns.access]), and the value contained in the object is the prvalue result. If the result is an erroneous value ([basic.indet]) and the bits in the value representation are not valid for the object's type, the behavior is undefined.

Insert a new subclause [9.12.?, dcl.attr.indet]:

> **9.12.? Indeterminate storage [dcl.attr.indet]**
>
> ?. The *attribute-token* `indeterminate` may be applied to the definition of a block variable with automatic storage duration or to a *parameter-declaration* of a function declaration. No *attribute-argument-clause* shall be present. The attribute specifies that the storage of an object with automatic storage duration is initially indeterminate rather than erroneous [6.7.4, basic.indet].
>
> ?. If a function parameter is declared with the `indeterminate` attribute, it shall be so declared in the first declaration of its function. If a function parameter is declared with the `indeterminate` attribute in the first declaration of its function in one translation unit and the same function is declared without the `indeterminate` attribute on the same parameter in its first declaration in another translation unit, the program is ill-formed, no diagnostic required.
>
> ?. [*Note*: Reading from an uninitialized variable that is marked `[[indeterminate]]` can cause undefined behavior.
>
> [*Example*:
>
> ```
> void f(int);
> void g() {
>   int x [[indeterminate]], y;
>   f(y);       // erroneous behavior [basic.indet]
>   f(x);       // undefined behavior
> }
>
> struct T { T(){} int x; };
> void h(T t = T()) {
>   f(t.x);    // undefined behavior if default argument is used
> }
> ```
>
> — *end example*]
>
> — *end note*]

Modify [11.9.3, class.base.init] paragraph 9:

> […] An attempt to initialize more than one non-static data member of a union renders the program ill-formed.

[*Note ?*: After the call to a constructor for class `X` for an object with automatic or dynamic storage duration has completed, if the constructor was not invoked as part of value-initialization and a member of `X` is neither initialized nor given a value during execution of the *compound-statement* of the body of the constructor, the member has an indeterminate or erroneous value ([basic.indet]). — *end note*]

[*Example ?*:

```
struct A {
  A();
};

struct B {
  B(int);
};

struct C {
  C() { }      // initializes members as follows:
  A a;         // OK, calls A::A()
  const B b;   // error: B has no default constructor
  int i;       // OK, i has indeterminate or erroneous value
  int j = 5;   // OK, j has the value 5
};
```

— *end example*]

Free up the term “erroneous” by modifying [7.6.2.8, expr.new] paragraph 8:

If the *expression* in a *noptr-new-declarator* is present, it is implicitly converted to `std::size_t`. The value of the *expression* is ~~erroneous~~invalid if:

- the expression is of non-class type and its value before converting to `std::size_t` is less than zero;
- the expression is of class type and its value before application of the second standard conversion ([over.ics.user])[footnote] is less than zero;
- its value is such that the size of the allocated object would exceed the implementation-defined limit ([implimits]); or
- the *new-initializer* is a *braced-init-list* and the number of array elements for which initializers are provided (including the terminating `'\0'` in a *string-literal* ([lex.string])) exceeds the number of elements to initialize.

If the value of the *expression* is ~~erroneous~~invalid after converting to `std::size_t`:

- if the *expression* is a potentially-evaluated core constant expression, the program is ill-formed;
- otherwise, an allocation function is not called; instead
  - if the allocation function that would have been called has a non-throwing exception specification ([except.spec]), the value of the *new-expression* is the null pointer value of the required result type;
  - otherwise, the *new-expression* terminates by throwing an exception of a type that would match a handler ([except.handle]) of type `std::bad_array_new_length` ([new.badlength]).

When the value of the *expression* is zero, the allocation function is called to allocate an array with no elements.

Free up the term “erroneous” by modifying [9.4.2, dcl.init.aggr] paragraph 16, which uses the term in the same sense as in the preceding edit, and we need to make that connection clear:

> Braces can be elided in an *initializer-list* as follows. If the *initializer-list* begins with a left brace, then the succeeding comma-separated list of *initializer-clause*s initializes the elements of a subaggregate; it is ~~erroneous~~invalid for there to be more *initializer-clause*s than elements.

Modify [16.4.4.4, nullablepointer.requirements] paragraph 2:

> A value-initialized object of type `P` produces the null value of the type. The null value shall be equivalent only to itself. A default-initialized object of type `P` may have an indeterminate or erroneous value.
>
> [*Note ?*: Operations involving indeterminate values can cause undefined behavior, and operations involving erroneous values can cause erroneous behavior. — *end note*]

Modify the specification of `std::bit_cast` to account for erroneous values by modifying [22.15.3, bit.cast] paragraph 2:

> *Returns*: An object of type `To`. Implicitly creates objects nested within the result ([intro.object]). Each bit of the value representation of the result is equal to the corresponding bit in the object representation of `from`. Padding bits of the result are unspecified. For the result and each object created within it, if there is no value of the object's type corresponding to the value representation produced, the behavior is undefined. If there are multiple such values, which value is produced is unspecified. A bit in the value representation of the result is indeterminate if it does not correspond to a bit in the value representation of `from` or corresponds to a bit of an object that is not within its lifetime or has an indeterminate value ([basic.indet]).
>
> For each bit in the value representation of the result that is indeterminate, the smallest object containing that bit has an indeterminate value; the behavior is undefined unless that object is of unsigned ordinary character type or `std::byte` type. The result does not otherwise contain any indeterminate values. If a bit of the result corresponds to a bit in the object representation of `from` that has an erroneous value, and the bits in the value representation for the smallest object `0` containing that bit are not valid for the type of `0`, the behavior is undefined (see also [conv.lval]); otherwise the behavior is erroneous, and the result is as specified above, where the value of `0` is erroneous.

Free up the term “erroneous” by modifying [33.7.3, thread.condition.nonmember] paragraph 5:

> [*Note 2*: It is the user’s responsibility to ensure that waiting threads do not ~~erroneously~~wrongly assume that the thread has finished if they experience spurious wakeups. [...] — *end note*]

Free up the term “erroneous” by modifying [C.6.6, diff.dcl] paragraph 6:

> **Rationale:** This is to avoid ~~erroneous function calls (i.e., ~~function calls with the wrong number or type of arguments~~)~~.

Impact and implementability

Applying erroneous behaviour to the default initialization of automatic variables is already available today. Clang and GCC expose an example of the new production behaviour when given the flag `-ftrivial-auto-var-init=zero`, with the caveat that this never exhibits the encouraged behaviour of diagnosing an error. Clang exposes the error-detecting behaviour when using its Memory Sanitizer (which currently detects undefined behaviour, and would have to be taught to also recognize erroneous behaviour).

The proposal primarily constitutes a change of the specification tools that we have available in the Standard, so that we have a formal concept of incorrect code that the Standard itself can talk about. It should pose only a minor implementation burden. Generally, the impact of changing an operation's current undefined behaviour to erroneous behaviour is as follows:

- On correct code: none observable, but possible performance cost.
- On incorrect code: if the code leads to previously undefined behaviour that is changed to erroneous behaviour, the code is still incorrect, but the behaviour is now as specified in the change, not unconstrained.

The broader picture: Erroneous behaviour in C++

The current C++ Standard speaks only about well-defined and well-behaved programs, and imposes no requirements on any other program. This results in an overly simple dichotomy of a program either being correct as written, with specified behaviour, or being incorrect and entirely outside the scope of the Standard. It is not possible for program to be incorrect, yet have its behaviour constrained by the Standard.

The newly proposed *erroneous behaviour* fills this gap. It is well-defined behaviour that is nonetheless acknowledged as being "incorrect", and thus allows implementations to offer helpful diagnostics, while at the same time being constrained by the specification.

Adopting erroneous behaviour for a particular operation consists of replacing current undefined behaviour with a (well-defined) specification of that operation's behaviour, explicitly called out as "erroneous". This will in general have a performance cost, and we need some *principles* for when we change a particular construction to have erroneous behaviour. We propose the following.

Principles of erroneous behaviour:

- Currently well-defined behaviour should not generally be made erroneous, since that would effectively break existing code. We can consider this in exceptional circumstances if a particular behaviour turns out to be always wrong.
- Currently undefined behaviour whose potential for harmful compilation is low, e.g. as evidenced by CVEs or other reported exploits of vulnerabilities, should generally remain undefined. This requires the least amount of complexity in the standard.
- Currently undefined behaviour which exposes harmful failure modes and for which we can find a reasonable well-defined behaviour should be considered for conversion to erroneous behaviour.
- Note: some kinds of undefined behaviour, even if harmful, cannot reasonably be detected in every case, and thus it would be infeasible to attempt to define the behaviour.
- Caution: erroneous behaviour is somewhat more likely to be wilfully relied upon by programmers than undefined behaviour, so we should err on the side of retaining undefined behaviour unless there is some clear benefit from defining the behaviour.

To present further examples, we consult the Shafik Yaghmour's P1705R1, which lists occurrences of undefined behaviour in the Standard. We pick a selection of cases and comment on whether each one might be a candidate for conversion to erroneous behaviour.

UB in C++23	Action	Comment
lexing splice ([lex.phases]) results in universal character name	leave as is	obscure, low potential for harm

UB in C++23	Action	Comment
Modifying a const value	leave as is	unlikely to happen (requires explicit, suspicious code), infeasible to specify behaviour
ODR violation	leave as is	infeasible
Read of indeterminate value	change to erroneous	(That is this paper!)
Signed integer overflow	could be changed to erroneous	The result of an overflowing operation could "erroneously be [some particular value]". This is not an uncommon bug. We consider it of low importance, though, since it is not a major safety concern.
Unrepresentable arithmetic conversions	could be changed to erroneous	Same as for signed integer overflow.
Bad bitshifts	could be changed to erroneous	Same as for signed integer overflow.
calling a function through the wrong function type	leave as is	uncommon, infeasible
invalid down-cast	leave as is	infeasible
invalid pointer arithmetic or comparison	leave as is	infeasible
invalid cast to enum	unsure	This needs investigation. Perhaps the invalid value could be erroneously preserved. Unclear if this would be useful.
various misuses of `delete`	leave as is	infeasible
type punning, union misuse, overlapping object access	leave as is	infeasible
null pointer dereference, null pointer-to-member dereference)	practically, leave as is	One could entertain a change to make a null pointer dereference erroneous, but the choice of behaviour is tricky. For scalars, the result could be some fixed value. Alternatively, the result could be termination. This would of course have a cost.
division by zero	could be changed to erroneous	Could erroneously result in some fixed value. The impact in the status quo is unclear; the change would have a cost.
flowing off the end of a non-`void` function; returning from a `[[noreturn]]` function	could be changed to erroneous	E.g. could erroneously call `std::terminate`. Mild additional cost. Unclear how valuable.
recursively entering the initialization of a block-static variable	unsure	Seems obscure.
accessing an object outside its lifetime	leave as is	infeasible

UB in C++23	Action	Comment
calling a pure-virtual function in an abstract base {con,de}structor	could be changed to erroneous	E.g. some particular pure-virtual handler could be called erroneously. This might already be the case on some implementations.
[class] doing things with members before construction has finished	leave as is	infeasible
data races	leave as is	infeasible (also not often a cause of vulnerabilities)
library undefined behaviour	case by case	A language-support facility such as "`std::erroneous()`" (which erroneously has no effect) could be used to allow for user-defined erroneous behaviour.
(speculative) contract violation	could be erroneous	Current work on contracts comes up against the question of what should happen in case of a contract violation. The notion of erroneous behaviour might provide a useful answer.

Tooling

While we have been emphasising the importance of code readability and understandability, we must also consider the practicalities of actually compiling and running code. Whether code has meaning, and if so, which, impacts tools. There are two important, and sometimes opposed, use cases we would like to consider.

Production compilers

Getting code to run in production often comes with two important (and also opposed) expectations:

- Performance: code should use as few resources as possible to achieve its specified behaviour.
- Safety: incorrect code should not have harmful side effects.

Undefined behaviour, and in particular its implications on the meaning of code, is increasingly exploited by compilers to optimise code generation. By assuming that undefined behaviour can never have been intentional, transitive assumptions can be derived that allow for far-reaching optimisations. This is often desirable and beneficial for correct code (and demonstrates the value of unambiguously understandable code: even compilers can use this reasoning to determine how much work does and does not have to be done). However, for incorrect code this can expose vulnerabilities, and thus constitute a considerable lack of safety. P1093R0 discusses these performance implications of undefined behaviour.

The proposed erroneous behaviour retains the same meaning of code as undefined behaviour for human readers, but the compiler has to accept that erroneous behaviour can happen. This constrains the compiler (as it as to ensure erroneous results are produced correctly), but in the event of incorrect code (which all erroneous behaviour requires), the resulting behaviour is constrained by the Standard and does not create a safety hazard. In other words, erroneous behaviour has a potential performance cost compared to undefined behaviour, but is safer in the presence of incorrect code.

Debug toolchains and sanitizers

The other major set of tools that software projects use are debugging tools. Those include extra warnings on compilers, static analysers, and runtime sanitizers. The former two are good at catching some localised bugs early, but do not catch every bug. Indeed one of the main limitations we seem to be discovering is that there is reasonable C++ code for which important analyses cannot be performed statically. (Note that P2687R0 proposes a safety strategy in which static analysis plays a major role.) Runtime sanitizers like ASAN, MSAN, TSAN, UBSAN, on the other hand, have excellent abilities to detect undefined behaviour at runtime with virtually no false positives, but at a significant build and runtime cost.

Both runtime sanitizers and static analysis can use the code readability signal from both undefined and erroneous behaviour equally well. In both cases it is clear that the code is incorrect. For undefined behaviour, implementations are unconstrained anyway and tools may reject or diagnose at runtime. The goal of erroneous behaviour is to permit the exact same treatment, by allowing a conforming implementation to diagnose, terminate (and also reject) a program that contains erroneous behaviour.

In other words, erroneous behaviour retains the understandability and debuggability of undefined behaviour, but also constrains the implementation just like well-defined behaviour.

Usage profiles

The following toolchain deployment examples are based on real-world setups.

- A safety-critical production system (e.g. one that handles user data) compiles code treating erroneous behaviour as well-defined, not issuing diagnostics. Compared to the status quo, the resulting executable is safer since erroneous behaviour is well-defined. There may be a flag to ensure that no erroneous behaviour is rejected, so as to not be blocked by erroneous but dead code.
- A safety-noncritical, high-performance system (e.g. a scientific simulation) may use a toolchain that assumes that no erroneous behaviour exists (similar to `-ffast-math`, which assumes that all floating point values lie in a certain range). This allows for best-possible performance, and corresponds to the status quo. Debugging and testing are performed separately.
- Debugging and testing deployments, including unit tests, use runtime sanitizers to detect undefined and erroneous behaviour. This is a continuous part of a larger production ecosystem, where test coverage and fuzzing tools try to expose as much of the production code to sanitizers as possible, and it is cultivated as part of the overall production effort.

Design alternatives for the opt-out mechanism

When it comes to an explicit syntax that restores the initialization behaviour prior to this proposed change, multiple plausible options exist. Paper P2723R1 proposed several options, which were polled in Issaquah. We will not repeat the details here, and only briefly summarise some of the options:

- an attribute (like proposed here, `int x [[foo]];`)
- a keyword (like “`int x = noinit;`”)
- a magic library type (like “`int x = std::uninitialized;`”)

In the Issaquah polling, the attribute (8|20|13|10|5) and keyword (3|11|10|18|11) options did not reach consensus, and the magic library type option (20|11|10|11|5) received only mild favours. Nonetheless, in this proposal we are again using an attribute, since the alternatives are significantly more novel and lack any form of implementation experience, whereas by contrast we have plenty of experience with attributes. An attribute is compatible with (i.e. can be deployed in code written as) previous versions of C++ down to C++11. The attribute meets the rules of ignorability: ignoring the opt-out does not change the meaning of a program that is correct *with* the opt-out.

Recall the word of caution from the design discussion: The mechanism to opt out of the safety feature must not be easily mistakable for a mechanism to annotate that an initialization is deliberately omitted. We will keep this in mind as it applies to the names chosen both for the novel syntax proposals above and also for the attribute, and we will dub this the “accidental self-documentation trap”.

A number of names had been brought forward for the opt-out attribute. We list only a few most plausible ones:

- × `uninitialized`: falls into the accidental self-documentation trap, as discussed in the design section
- × `noinit`: quite similarly subject to the accidental self-documentation trap (and shows that a `noinit` keyword would also be problematic for the same reasons)
- × `for_overwrite`: quite similarly subject to the accidental self-documentation trap, though a nice connection with existing for-overwrite facilities
- × `no_trivial_initialization`: this is a bit confusing, since one can read “initialization” in multiple ways
- × `not_zeroed`: too specific and tied to implementation details; we are deliberately allowing erroneous values to vary at the implementation’s discretion, and they are not required to be zero
- × `possible_security_hole`: too cute and imprecise, but demonstrates how this is about disabling a safety mechanism
- × `assume_indeterminate`: close, but perhaps needlessly verbose
- ✔ `indeterminate`: short enough, matches the core wording, and mysterious enough to require users to look up how it works instead of assuming

What is code?

Here follows a high-level position the meaning of code, which underpins my motivation to preserve the ability to recognize code as incorrect even if it has well-defined behaviour.

Code is communication. Primarily, code communicates an idea *among humans*. Humans work with code as an evolving and accumulating resource. Its role in software engineering projects is not too different from the role of traditional literature in the pursuit of science, technology, and engineering: literature is how individuals learn from and contribute to collective progress. The fact that code can also be interpreted and executed by computers is of course also important, but secondary. (There are many ways one can instruct a machine, but not all of them are suitable for building a long-term ecosystem.)

The language of code are programming languages, and the medium is source code, just like natural languages written in books, emails, or spoken in videos are the media of traditional literature. Like all media, source code is imperfect and ambiguous. The purpose of a text is to communicate an idea, but the entire communication has to be funnelled through the medium, and understood by the audience. Without the author present to explain what they really meant, the text is the only clue to the original idea; any act of reading a text is always an act of forensic reconstruction of the original idea. If the text is written well and “clear”, then readers can perform this reconstruction with high confidence that they “got it right” and feel themselves understanding the idea; they are “on the same page” as the author. On the other hand, poor writing leads to ambiguous text, and reading requires interpretation and often guess-work. This is no different in natural languages than in computer code.

I would like to propose that we appreciate the value of code *as communication with humans*, and consider how well a programming language works for that purpose in medium of source code. Source code is often shared among a large group of users, who are actively working with the code: code is only very rarely a complete black-box that can be added to a project without further thought. At the very least, interfaces and vocabulary have to be understood. But commonly, too, code has to be modified in order to be integrated into a project, and to be evolved in response to new requirements. Last but not least, code often contains errors, which have to be found, understood, and fixed. All of the above efforts may be performed by a

diverse group of users, none of whom need to have intimate familiarity with any one piece of code. There is value in having *any* competent users be able to read and understand any one piece of code — not necessarily in all its domain depth, but well enough to work with it in the context of a larger project. To extent the analogy with natural language above, this is similar to how a competent speaker of a language should be able to understand and integrate a well-made argument in a discussion, even if they are not themselves an expert in the domain of the argument.

How does all this connect to C++? Like with code in any programming language, given a piece of code, a user should be able to understand the idea that the code is communicating. Absent a separate document that says “Here is what this code is meant to do:”, the main source of information available to the user is the behaviour of the code itself. Note how this has nothing to do with compiling and running code. At this point, the code and the idea it communicates exist only in the minds of the author and the reader; no compilation is involved. How well the user understands the code depends on how ambiguous the code is, that is, how many different things it can mean. The user interprets the code by choosing a possible meaning to it from among the choices, where we assume that the code is *correct*: in C++, that means correct in the sense of the Standard, being both well-formed and executing with well-defined behaviour. This is critical: the constraint of presumed correctness serves as a dramatic aid for interpretation. If we assume that code is correct, then we can dismiss any interpretation that would require incorrect behaviour, and we have to decide among only the few remaining valid interpretations. The more valid interpretations a construction has, the more ambiguity a user has during interpretation of the entire piece of code. C++ defines only a very narrow set of behaviours, and everything else is left as the infamous *undefined behaviour*, which we could say is not C++ at all, in the sense that we assume that that's not what could possibly have been meant. Practically, of course, we would not dismiss undefined behaviour as “not C++”, but instead we would treat it as a definitive signal that the code is not communicating its idea correctly. (We could then either ask the author for clarification, or, if we are confident to have understood the correct idea anyway, we can fix the code to behave correctly. I claim that in this long-term perspective on code as a cultural good, buggy code with a clear intention is better than well-behaved, ambiguous code: if the intention is clear, then I can see if the code is doing the right thing and fix it if not, but without knowing the intention, I have no idea if the well-behaved code is doing what it is supposed to.)

Related work

Sean Parent’s presentation *Reasoning About Software Correctness* (and also subsequent Cpp North 2022 keynote talk) give a useful definition of “safety” adapted specifically to C++ and which explicitly concerns only *incorrect code*. He defines a function to be safe if it does not lead to undefined behaviour (that is, even when preconditions are violated).

JF Bastien's paper P2723R1 proposes addressing the safety concerns around automatic variable initialization by just defining variables to be initialized to zero. The previous revision of that paper was what motivated the current proposal: The resulting behaviour is desirable, but the cost on code understandability is unacceptable to the present author.

The papers P2687R0 by Bjarne Stroustrup and Gabriel Dos Reis and P2410R0 by Bjarne take a more general look at how to arrive at a safe language. They recommends a combination of static analysis and restrictions on the use of the language so as to make static analysis very effective. However, on the subject of automatic variable initialization specifically they offers no new solution: P2687R0 recommends only either zero-initialization or annotated non-initialization (reading of which results in UB); in that regard it is similar to JF Bastien's proposal. P2410R0 states that “[s]tatic analysis easily prevents the creation of uninitialized objects”, but the intended result of this prevention, and in particular the impact on code understandability, is left open.

Tom Honerman proposed a system of “diagnosable events”, which is largely aligned with the values and goals of this proposal, and takes a quite similar approach: Diagnosable events have well-defined behaviour, but implementations are permitted to handle them in an implementation-defined way.

Davis Herring's paper P1492R2 proposes a checkpointing system that would stop undefined behaviour from having arbitrarily far-reaching effects. That is a somewhat different problem area from the present safety one, and in particular, it does not control the effects of the undefined behaviour itself, but merely prevents it from interfering with other, previous behaviour. (E.g. this would not prevent the leaking of secrets via uninitialized variables.)

The Ada programming language has a notion of bounded undefined behaviour.

Paper P1093R0 by Bennieston, Coe, Gahir and Russel discusses the value of undefined behaviour in *correct* code and argues for the value of the compiler optimizations that undefined behaviour permits. This is essentially the tool’s perspective of the value of undefined behaviour for the interpretability of code which we discussed above: both humans and compilers benefit from being able to understand code with fewer ambiguities. Compilers can use the absence of ambiguities to avoid generating unnecessary code. The paper argues that we should not break these optimizations lightheartedly by making erstwhile undefined behaviour well-defined.

Questions and answers

Do you really mean that there can never be any UB in any correct code? There is of course always room for nuance and detail. If a particular construction is known to be UB, but still appropriate on some platform or under some additional assumptions, it is perfectly fine to use it. It should be documented/annotated sufficiently, and perhaps tools that detect UB need to be informed that the construction is intentional.

Why is static analysis not enough to solve the safety problem of UB? Why do we need sanitizers? Current C++ is not constrained enough to allow static analysis to accurately detect all cases of undefined behaviour. (For example, C++ allows initializing a variable via a call to a function in a separate translation unit or library.) Other languages like Rust manage to prevent unsafe behaviour statically, but they are more constrained (e.g. Rust does not allow passing an uninitialized value to a function). Better static analysis is frequently suggested as a way to address safety concerns in C++ (e.g. P2410R0, P2687R0), but this usually requires adopting a limited subset of C++ that is amenable to reliable static analysis. This does not help with the wealth of existing C++ code, neither with making it safe nor with making it correct. By contrast, runtime sanitizers can reliably point out when undefined behaviour is reached.

Why is `int x;` any different from `std::vector<int> v;`? Several reasons. One is that `vector` is a class with internal invariants that needs to be destructible, so a well-defined initial state already suggests itself. The other is that a vector is a container of elements, and if the initializer does not provide any elements, then a vector with no elements is an unsurprising result. By contrast, if there is no initial value given for an `int`, there is no single number that is better or more obviously right than any other number. Zero is a common choice in other languages, but it does not seem helpful in the sense of making it easy to write unambiguous code if we allow a novel spelling of a zero-valued `int`. If you mean zero, just say `int x = 0;`.

Is this proposal better than defining `int x;` to be zero? It depends on whether you want code to deliberately use `int x;` to mean, deliberately, that `x` is zero. The counter-position, shared by this author, is that zero should have no such special treatment, and all initialization should be explicit, `int x = -1, y = 0, z = +1;`. All numeric constants are worth seeing explicitly in code, and there is no reason to allow `int x;` as a valid alternative for one particular case that already has a perfectly readable spelling. (An explicit marker for a deliberately uninitialized variable is still a good idea, and accessing such a variable would remain undefined behaviour, and not become erroneous even in this present proposal.)

Acknowledgements

Many thanks to Loïc Joly for extensive help on restructuring and refocussing this document, to Richard Smith for discussion and wording, to Andrzej Krzemienski for wording suggestions and for pointing out a possible application to contracts, to Ville Voutilainen for naming suggestions, and to John Lakos for valuable feedback and support. Thanks for valuable discussions and encouragement also to JF Bastien and to members of SG23. Thanks to Jens Maurer for an in-depth wording review and reorganization. Finally, thanks to the members of CWG for many wording suggestions.

References

- Andrew Bennieston, Jonathan Coe, Daven Gahir, Thomas Russel, *P1093R0: Is undefined behaviour preserved?*
- Tom Honerman, *Posioned values: A feature and specification mechanism to aid diagnosis of implicitly initialized variables*, private communication.
- Davis Herring, *P1494R2: Partial program correctness*.
- JF Bastien, *P2723R1: Zero-initialize objects of automatic storage duration*.
- Sean Parent, *Reasoning About Software Correctness*.
- Sean Parent, Cpp North 2022 keynote talk, *The Tragedy of C++*.
- Bjarne Stroustrup, Gabriel Dos Reis, *P2687R0: Design Alternatives for Type-and-Resource Safe C++*.
- Bjarne Stroustrup, *P2410R0: Type-and-resource safety in modern C++*.
- Jake Fevold, *P2754R0: Deconstructing the Avoidance of Uninitialized Reads of Auto Variables*.
- Shafik Yaghmour, *P1705R1: Enumerating Core Undefined Behavior*.
- Jonathan Müller P0632R0: *Proposal of [[uninitialized]] attribute*